

좋아. 지금 네 1주차 Todo API에서 실제로 등장하는 용어들을, “정의 + 어디서 쓰는지 + 무엇과 구분해야 하는지” 기준으로 전부 묶어서 정리해볼게. 이 프로젝트는 DB/JPA/Security 없이 Controller, Service, Repository, DTO, Domain, Exception 구조를 익히는 학습용 프로젝트야.

 README

## 1. 프로젝트 실행과 구조 용어

| 용어                        | 정의                                       | 네 프로젝트에서의 의미                                                          |
|---------------------------|------------------------------------------|-----------------------------------------------------------------------|
| Spring Boot               | Spring 기반 애플리케이션을 쉽게 실행하고 설정하게 해주는 프레임워크 | Todo API 서버 실행                                                        |
| Spring Initializr         | Spring 프로젝트의 기본 뼈대를 만들어주는 도구             | <code>build.gradle</code> , <code>gradlew</code> , Application 클래스 생성 |
| Gradle                    | 프로젝트 빌드·실행·테스트·의존성 관리를 하는 도구             | <code>bootRun</code> , <code>test</code> 실행                           |
| <code>gradlew</code>      | 프로젝트에 포함된 Gradle 실행 파일                   | macOS/Linux용                                                          |
| <code>gradlew.bat</code>  | Windows용 Gradle 실행 파일                    | <code>.\gradlew.bat bootRun</code>                                    |
| Dependency                | 프로젝트가 사용하는 외부 라이브러리                      | Spring Web, Validation, Lombok                                        |
| <code>build.gradle</code> | Gradle 설정 파일                             | 라이브러리, Java 버전, 플러그인 설정                                               |
| Package                   | Java 클래스를 분류하는 경로                        | <code>controller</code> , <code>service</code> , <code>dto</code>     |
| Application class         | 프로그램 시작점                                 | <code>SpringApplication.run(...)</code> 실행                            |
| Server                    | HTTP 요청을 기다리고 응답하는 프로그램                  | Spring Boot 내장 서버                                                     |
| Port                      | 서버가 요청을 받는 번호                            | 기본값 <code>8080</code>                                                 |
| localhost                 | 현재 내 컴퓨터를 의미하는 주소                        | <code>http://localhost:8080</code>                                    |

## 2. HTTP와 API 용어

| 용어                        | 정의                             | 예시                                           |
|---------------------------|--------------------------------|----------------------------------------------|
| HTTP                      | 프론트엔드와 백엔드가 통신할 때 사용하는 규칙      | GET, POST 요청                                 |
| API                       | 프로그램이 다른 프로그램과 통신하기 위한 인터페이스   | <code>/api/todos</code>                      |
| Endpoint                  | 특정 기능을 제공하는 API 주소             | <code>GET /api/todos</code>                  |
| URL                       | 요청을 보내는 전체 주소                  | <code>http://localhost:8080/api/todos</code> |
| URI                       | 서버 안에서 자원을 식별하는 경로             | <code>/api/todos/1</code>                    |
| Request                   | 클라이언트가 서버에 보내는 메시지             | Todo 생성 요청                                   |
| Response                  | 서버가 클라이언트에 돌려주는 메시지            | Todo JSON 응답                                 |
| Request body              | 요청에 담아 보내는 실제 데이터              | title, description                           |
| Response body             | 서버가 돌려주는 데이터                   | id, completed 등                              |
| Header                    | 요청·응답에 대한 부가 정보                | <code>Content-Type</code>                    |
| <code>Content-Type</code> | 전송 데이터 형식을 나타냄                 | <code>application/json</code>                |
| JSON                      | 데이터를 문자열 형태로 표현하는 형식           | <code>{ "title": "공부" }</code>               |
| Status code               | 요청 결과를 숫자로 표현                  | 200, 201, 400, 404                           |
| REST API                  | 자원 중심으로 HTTP 메서드를 사용하는 API 스타일 | <code>/api/todos/{id}</code>                 |

### 3. HTTP 메서드

| 메서드  | 의미    | Todo API |
|------|-------|----------|
| GET  | 조회    | 목록·단건 조회 |
| POST | 생성    | Todo 생성  |
| PUT  | 전체 교체 | 현재 미사용   |

| 메서드    | 의미    | Todo API     |
|--------|-------|--------------|
| PATCH  | 일부 수정 | 제목 수정, 완료 처리 |
| DELETE | 삭제    | Todo 삭제      |

중요한 차이:

POST

→ 새로운 Todo 생성

PATCH

→ 기존 Todo의 일부 변경

DELETE

→ 기존 Todo 삭제



## 4. HTTP 상태 코드

| 상태 코드 | 이름                    | 의미             |
|-------|-----------------------|----------------|
| 200   | OK                    | 정상 조회·수정       |
| 201   | Created               | 새로운 데이터 생성 성공  |
| 204   | No Content            | 성공했지만 응답 본문 없음 |
| 400   | Bad Request           | 요청값이 잘못됨       |
| 404   | Not Found             | 요청 대상이 없음      |
| 500   | Internal Server Error | 서버 내부 오류       |

예를 들어:

POST /api/todos 성공

→ 201 Created

DELETE /api/todos/1 성공

→ 204 No Content

빈 title

→ 400 Bad Request

없는 id 조회

→ 404 Not Found



README에도 Validation 실패는 400, 존재하지 않는 Todo는 404로 구분되어 있어.

 README

## 5. Spring Bean 관련 용어

| 용어                    | 정의                                  |
|-----------------------|-------------------------------------|
| Bean                  | Spring이 생성하고 관리하는 객체                |
| Bean 등록               | 어떤 객체를 Spring 관리 대상으로 만드는 것         |
| Bean container        | Bean들을 생성·보관·연결하는 Spring 내부 공간      |
| ApplicationContext    | Spring Bean 컨테이너의 대표 인터페이스          |
| Component Scan        | 특정 패키지를 탐색해 Bean을 자동 등록하는 기능        |
| Dependency Injection  | 필요한 객체를 외부에서 넣어주는 방식                |
| Constructor Injection | 생성자를 통해 의존성을 주입하는 방식                |
| IoC                   | 객체 생성과 제어를 개발자가 아니라 Spring이 담당하는 개념 |

네 프로젝트에서는 대략 이렇게 연결돼.

```
TodoController Bean
→ TodoService Bean 주입

TodoService Bean
→ InMemoryTodoRepository Bean 주입
```



## 6. Bean 등록 애너테이션

| 애너테이션           | 의미                 |
|-----------------|--------------------|
| @Component      | 일반적인 Bean 등록       |
| @Service        | 서비스 계층 Bean        |
| @Repository     | 저장소 계층 Bean        |
| @Controller     | MVC 화면 컨트롤러 Bean   |
| @RestController | JSON API 컨트롤러 Bean |
| @Configuration  | 설정 클래스 Bean        |

## 애너테이션

## 의미

@Bean

메서드 반환 객체를 직접 Bean으로 등록

@RestControllerAdvice

전역 REST 예외 처리 Bean

## 중요한 구분:

@Service

→ 클래스를 Bean으로 등록

@PostMapping

→ POST 요청을 메서드에 연결



@PostMapping 은 Bean이 아니야.

## 7. 의존성 주입 용어

### 용어

### 정의

Dependency

어떤 클래스가 사용하기 위해 필요한 다른 객체

Injection

필요한 객체를 외부에서 전달하는 것

Constructor injection

생성자를 통해 전달

Interface injection

인터페이스 타입으로 구현체를 주입받는 방식

Loose coupling

구체 구현에 덜 의존하는 구조

Implementation

인터페이스에 정의된 기능을 실제로 구현한 클래스

예:

<> Java

```
private final TodoRepository todoRepository;
```



TodoService 는 InMemoryTodoRepository 를 직접 적지 않고 TodoRepository 인터페이스에 의존해.

## 8. Lombok 용어

### 용어

### 정의

Lombok

반복적인 Java 코드를 자동 생성해주는 라이브러리

| 용어                                    | 정의                                   |
|---------------------------------------|--------------------------------------|
| <code>@RequiredArgsConstructor</code> | <code>final</code> 필드를 받는 생성자를 자동 생성 |
| Getter                                | 필드 값을 읽는 메서드                         |
| Setter                                | 필드 값을 변경하는 메서드                       |
| Constructor                           | 객체를 생성할 때 호출되는 메서드                   |

예:

Java

```
@RequiredArgsConstructor
public class TodoController {
    private final TodoService todoService;
}
```

실제로는 Lombok이 생성자를 만들어준다고 보면 돼.

## 9. 애너테이션

| 용어                    | 정의                         |
|-----------------------|----------------------------|
| Annotation            | 클래스·메서드·필드 등에 의미를 추가하는 표시  |
| Runtime annotation    | 실행 중 Spring이 읽어 처리하는 애너테이션 |
| Mapping annotation    | HTTP 요청을 메서드에 연결하는 애너테이션   |
| Validation annotation | 입력값 검증 규칙을 표시하는 애너테이션      |

애너테이션 자체는 `@`로 시작해.

Java

```
@RestController
@PostMapping
@Valid
```

## 10. Controller 용어

| 용어         | 정의             |
|------------|----------------|
| Controller | HTTP 요청을 받는 입구 |

| 용어                | 정의                          |
|-------------------|-----------------------------|
| REST Controller   | JSON 요청·응답을 처리하는 Controller |
| Request mapping   | URL과 메서드를 연결하는 것            |
| Handler method    | 실제 요청을 처리하는 Controller 메서드  |
| Parameter binding | 요청값을 Java 매개변수로 연결하는 과정     |
| Serialization     | Java 객체를 JSON으로 변환          |
| Deserialization   | JSON을 Java 객체로 변환           |

네 프로젝트의 Controller 역할:

```

HTTP 요청 받기
→ 요청 DTO 생성
→ Validation
→ Service 호출
→ 응답 DTO 반환

```



## 11. Controller 애너테이션

| 애너테이션           | 의미                   |
|-----------------|----------------------|
| @RestController | API Controller 선언    |
| @RequestMapping | 공통 URL 설정            |
| @GetMapping     | GET 요청 연결            |
| @PostMapping    | POST 요청 연결           |
| @PatchMapping   | PATCH 요청 연결          |
| @DeleteMapping  | DELETE 요청 연결         |
| @RequestBody    | JSON 요청을 Java 객체로 변환 |
| @PathVariable   | URL 경로값을 변수로 받음      |
| @RequestParam   | 쿼리 문자열 값을 받음         |
| @ResponseStatus | 응답 상태 코드 지정          |

## 애너테이션

## 의미

@Valid

DTO Validation 실행

예:

<> Java



```
@GetMapping("/{id}")
public TodoResponse findById(@PathVariable Long id)
```

```
GET /api/todos/1
→ id = 1
```



## 12. Path Variable과 Request Parameter

### 용어

### 설명

### 예시

Path Variable

URL 경로 자체에 포함된 값

/todos/1

Request Parameter

? 뒤에 붙는 조건값

/todos?completed=true

예:

<> Java



```
@PathVariable Long id
```

```
/api/todos/3
→ id = 3
```



<> Java



```
@RequestParam boolean completed
```

```
/api/todos?completed=true
→ completed = true
```



## 13. DTO 관련 용어

### 용어

### 정의

DTO

계층 또는 시스템 사이에서 데이터를 전달하는 객체



| 용어           | 정의                     |
|--------------|------------------------|
| Request DTO  | 클라이언트 요청을 담는 객체        |
| Response DTO | 서버 응답을 담는 객체           |
| Mapping      | 한 객체를 다른 객체로 변환하는 과정   |
| Record       | 데이터를 간단하게 표현하는 Java 문법 |
| Field        | 객체가 가지는 데이터 항목         |

네 프로젝트 DTO:

| DTO                            | 역할           |
|--------------------------------|--------------|
| <code>TodoCreateRequest</code> | 생성 요청        |
| <code>TodoUpdateRequest</code> | 수정 요청        |
| <code>TodoResponse</code>      | 정상 응답        |
| <code>ErrorResponse</code>     | 오류 응답        |
| <code>HelloResponse</code>     | Hello API 응답 |

DTO는 데이터 전달용, Domain은 실제 업무 객체야.

## 14. Domain 용어

| 용어            | 정의                    |
|---------------|-----------------------|
| Domain        | 실제 업무에서 다루는 대상과 규칙    |
| Domain object | 업무 상태와 행동을 가진 객체      |
| State         | 객체가 현재 가지고 있는 값       |
| Behavior      | 객체가 수행하는 행동           |
| Encapsulation | 상태 변경을 객체 내부에서 관리하는 것 |
| Business rule | 업무상 지켜야 하는 규칙         |

예:

```
todo.complete();
```

`completed` 값을 밖에서 직접 바꾸기보다 Todo 내부 행동으로 표현하는 방식이야.

## 15. Entity와 Domain의 차이

1주차에서는 JPA를 안 쓰지만 구분은 알아두면 좋아.

| Domain               | Entity                     |
|----------------------|----------------------------|
| 업무 개념과 규칙 중심         | DB 저장 대상 중심                |
| 반드시 DB와 연결될 필요 없음    | 보통 JPA와 DB에 연결             |
| 현재 <code>Todo</code> | 다음 주차에는 JPA Entity가 될 수 있음 |

## 16. Service 용어

| 용어             | 정의                        |
|----------------|---------------------------|
| Service        | 기능의 실행 순서와 비즈니스 로직을 담당    |
| Business logic | 실제 업무 규칙과 처리 흐름           |
| Use case       | 사용자가 수행하려는 하나의 기능         |
| Orchestration  | 여러 객체의 호출 순서를 조정하는 것      |
| Transaction    | 여러 DB 작업을 하나의 작업 단위로 묶는 것 |

현재는 DB가 없으므로 Transaction은 개념만 알아도 돼.

Service 예:

```
Todo 조회
→ 없으면 예외
→ 상태 변경
→ 응답 DTO 변환
```

## 17. Repository 용어

| 용어         | 정의                    |
|------------|-----------------------|
| Repository | 데이터 저장·조회 기능을 추상화한 계층 |
| Interface  | 어떤 기능이 필요한지 정의한 계약    |

| 용어                   | 정의                        |
|----------------------|---------------------------|
| Implementation class | 인터페이스 기능을 실제로 구현한 클래스     |
| In-memory repository | 메모리에 데이터를 저장하는 Repository |
| Persistence          | 데이터를 지속적으로 보관하는 것         |
| Data access          | 데이터를 저장·조회·수정·삭제하는 작업     |

현재 구조:

TodoRepository

→ 기능 정의



InMemoryTodoRepository

→ Map으로 실제 구현

## 18. In-memory와 Map

| 용어                           | 정의                                 |
|------------------------------|------------------------------------|
| In-memory                    | 데이터를 메모리에 저장하는 방식                  |
| <code>Map&lt;K, V&gt;</code> | Key와 Value 쌍으로 데이터를 저장하는 Java 자료구조 |
| Key                          | 데이터를 구별하는 값                        |
| Value                        | 실제 저장되는 객체                         |
| Sequence                     | id를 순서대로 생성하기 위한 값                 |

예:

Java

`Map<Long, Todo>`



1 → 첫 번째 Todo

2 → 두 번째 Todo



서버를 종료하면 데이터가 사라지는 이유는 메모리에만 저장하기 때문이야.

## 19. CRUD

| 문자 | 의미     | Todo 기능 |
|----|--------|---------|
| C  | Create | 생성      |
| R  | Read   | 조회      |
| U  | Update | 수정      |
| D  | Delete | 삭제      |

Todo API 전체는 CRUD 기능을 연습하기 위한 구조야.

## 20. Optional

| 용어                             | 정의                          |
|--------------------------------|-----------------------------|
| <code>Optional&lt;T&gt;</code> | 값이 있을 수도, 없을 수도 있음을 표현하는 객체 |
| <code>orElseThrow()</code>     | 값이 없으면 예외를 발생시키는 메서드        |
| <code>null</code>              | 값이 없음을 표현하지만 오류 위험이 큼       |

예:

Java

```
todoRepository.findById(id)
    .orElseThrow(() -> new TodoNotFoundException(id));
```

의미:

Todo가 있으면 반환  
없으면 `TodoNotFoundException` 발생

## 21. Exception 용어

| 용어               | 정의               |
|------------------|------------------|
| Exception        | 정상 흐름을 벗어난 문제 상황 |
| Throw            | 예외를 발생시키는 것      |
| Catch            | 발생한 예외를 처리하는 것   |
| Custom exception | 개발자가 직접 만든 예외    |

| 용어                       | 정의                         |
|--------------------------|----------------------------|
| Global exception handler | 여러 Controller의 예외를 한곳에서 처리 |
| Error response           | 클라이언트에 반환하는 오류 JSON        |

네 프로젝트 흐름:

Todo 없음

→ TodoNotFoundException  
→ GlobalExceptionHandler  
→ ErrorResponse  
→ 404



## 22. 예외 처리 애너테이션

| 애너테이션                 | 의미             |
|-----------------------|----------------|
| @RestControllerAdvice | 전역 예외 처리 클래스   |
| @ExceptionHandler     | 특정 예외 처리 메서드   |
| @ResponseStatus       | 예외 또는 응답 상태 지정 |

## 23. Validation 용어

| 용어            | 정의                          |
|---------------|-----------------------------|
| Validation    | 입력값이 규칙에 맞는지 확인             |
| Constraint    | 입력값에 적용되는 제한 규칙             |
| Binding error | 요청값을 객체에 연결하거나 검증할 때 발생한 오류 |
| Field error   | 특정 필드에서 발생한 검증 오류           |

대표 애너테이션:

| 애너테이션     | 의미                 |
|-----------|--------------------|
| @NotBlank | null, 빈 문자열, 공백 금지 |
| @NotNull  | null 금지            |
| @Size     | 길이 제한              |

@Valid

검증 실행

## 24. 테스트 용어

### 용어

### 정의

Test

코드가 예상대로 동작하는지 확인

Unit test

작은 단위의 기능 테스트

Integration test

여러 계층을 연결한 테스트

JUnit 5

Java 테스트 프레임워크

MockMvc

Spring MVC 요청을 테스트하는 도구

Assertion

실제 결과가 예상 결과와 같은지 확인

Test case

하나의 테스트 시나리오

Arrange

테스트 데이터 준비

Act

테스트 대상 실행

Assert

결과 검증

예:

빈 제목으로 POST 요청

→ 400인지 확인



## 25. Mock과 실제 객체

### 용어

### 정의

Mock

실제 객체처럼 동작하도록 만든 가짜 객체

Stub

정해진 값을 반환하는 테스트용 객체

MockMvc

HTTP 요청을 흉내 내는 Spring 테스트 도구

MockMvc는 실제 브라우저 없이 Controller 요청을 테스트할 수 있게 해줘.

## 26. 프론트엔드와 백엔드 연결 용어

| 용어       | 정의                                |
|----------|-----------------------------------|
| Frontend | 사용자 화면을 담당                        |
| Backend  | 데이터 처리와 서버 로직 담당                  |
| Client   | 서버에 요청을 보내는 쪽                     |
| Server   | 요청을 받아 처리하는 쪽                     |
| Fetch    | 브라우저에서 HTTP 요청을 보내는 JavaScript 함수 |
| Axios    | HTTP 요청을 보내는 라이브러리                |
| CORS     | 서로 다른 출처 간 요청을 제한하는 브라우저 정책       |
| Origin   | 프로토콜 + 도메인 + 포트 조합                |

예:

```
React: localhost:5173
Spring: localhost:8080
```



포트가 다르므로 CORS 문제가 생길 수 있어.

## 27. IntelliJ HTTP Client 용어

| 용어                      | 정의                        |
|-------------------------|---------------------------|
| <code>.http</code> file | HTTP 요청을 저장하고 실행하는 파일     |
| <code>{{HOST}}</code>   | HTTP Client 변수            |
| Request separator       | 요청을 구분하는 <code>###</code> |
| Environment variable    | 실행 환경별 주소 등을 저장하는 변수      |

예:

```
<> http

@HOST = http://localhost:8080

GET {{HOST}}/api/todos
```



이건 Spring 문법이 아니라 요청 테스트 도구 문법이야.

## 28. Serialization과 Deserialization

| 용어              | 정의                                   |
|-----------------|--------------------------------------|
| Serialization   | Java 객체를 JSON으로 변환                   |
| Deserialization | JSON을 Java 객체로 변환                    |
| Jackson         | Spring Boot에서 JSON 변환에 주로 사용하는 라이브러리 |

흐름:

```
프론트 JSON
→ Jackson
→ TodoCreateRequest

TodoResponse
→ Jackson
→ JSON 응답
```



## 29. 전체 요청 흐름 용어

Todo 생성 기준:

```
Client
→ HTTP Request
→ DispatcherServlet
→ Controller
→ Request DTO
→ Validation
→ Service
→ Domain
→ Repository
→ Response DTO
→ JSON Response
```



여기서 `DispatcherServlet`은 Spring MVC의 중앙 요청 처리기야. 지금은 “요청을 적절한 Controller로 보내주는 역할” 정도만 알면 충분해.

## 30. 이번 주 반드시 설명할 수 있어야 하는 핵심 용어

아래는 단순히 이름만 아는 것이 아니라, 네 말로 정의할 수 있어야 해.

```
Spring Bean
Dependency Injection
Controller
Service
Repository
```





DTO  
Domain  
HTTP Request  
HTTP Response  
Endpoint  
Path Variable  
Request Body  
Validation  
Exception  
Status Code  
CRUD  
Interface  
Implementation  
In-memory  
Serialization  
Deserialization

## 31. 아직 깊게 안 해도 되는 용어

JPA  
Hibernate  
EntityManager  
Persistence Context  
Lazy Loading  
N+1  
Transaction propagation  
Spring Security  
JWT  
OAuth  
Redis  
Kafka  
Docker  
CI/CD



지금 1주차에서는 이걸 억지로 미리 공부할 필요 없어.

## 최종 초압축 요약

Controller  
→ HTTP 요청을 받음

DTO  
→ 데이터를 전달함

Service  
→ 기능 흐름을 처리함

Domain  
→ 실제 Todo 상태와 행동을 표현함

Repository



→ 데이터를 저장하고 조회함

Bean

→ Spring이 관리하는 객체

Dependency Injection

→ 필요한 Bean을 다른 객체에 넣어줌

Validation

→ 요청값을 검사함

Exception Handler

→ 예외를 HTTP 오류 응답으로 바꿈

HTTP Method

→ GET 조회, POST 생성, PATCH 수정, DELETE 삭제

이 용어들을 코드에서 찾아서 “이건 무슨 역할인지” 즉시 말할 수 있으면 1주차는 충분히 잘 따라가고 있는 거야.